

Algorithms_Data Structures

Exercise : E-commerce Platform Search Function

1.Explain Big O notation and how it helps in analyzing algorithms.

Big O notation used to measure the execution time of algorithm. It help to analyze how efficient the algorithm works.

2. Describe the best, average, and worst-case scenarios for search operations.

Best case: if element found in 0th index $O(1)$

Average case: if the element in middle $O(n)$

Worst case: if the element is in last $O(n)$

3.Implementation:

```
public class Product {  
    private int productId;  
    private String productName;  
    private String category;  
    public Product(int productId, String productName, String category) {  
        this.productId = productId;  
        this.productName = productName;  
        this.category = category;  
    }  
    public int getProductId() {  
        return productId;  
    }  
    public String getProductName() {  
        return productName;  
    }  
    public String getCategory() {  
        return category;  
    }  
    public void display() {  
        System.out.println(  
            "ID: " + productId +  
            ", Name: " + productName +  
            ", Category: " + category  
        );  
    }  
}
```

```
public class LinearSearch {  
    public static Product search(Product[] products, int productId) {  
        for (Product product : products) {  
            if (product.getProductId() == productId) {  
                return product;  
            }  
        }  
        return null;  
    }  
}
```

```

1 public class BinarySearch {
2
3     public static Product search(Product[] products, int productId) {
4
5         int low = 0;
6         int high = products.length - 1;
7
8         while (low <= high) {
9
10            int mid = (low + high) / 2;
11
12            if (products[mid].getProductId() == productId) {
13                return products[mid];
14            }
15
16            if (products[mid].getProductId() < productId) {
17                low = mid + 1;
18            } else {
19                high = mid - 1;
20            }
21        }
22
23        return null;
24    }
25 }

```

```

Week_1 > Design_and_principal > SearchTest.java > SearchTest > main(String[])
1 public class SearchTest {
2
3     public static void main(String[] args) {
4
5         Product[] products = {
6             new Product(productId: 101, productName: "Laptop", category: "Electronics"),
7             new Product(productId: 102, productName: "Phone", category: "Electronics"),
8             new Product(productId: 103, productName: "Watch", category: "Accessories"),
9             new Product(productId: 104, productName: "Tablet", category: "Electronics"),
10            new Product(productId: 105, productName: "Camera", category: "Electronics")
11        };
12
13        System.out.println(x: "Linear Search:");
14
15        Product result1 =
16            LinearSearch.search(products, productId: 104);
17
18        if (result1 != null) {
19            result1.display();
20        }
21
22        System.out.println();
23
24        System.out.println(x: "Binary Search:");
25
26        Product result2 =
27            BinarySearch.search(products, productId: 104);
28
29        if (result2 != null) {
30            result2.display();
31        }
32
33    }

```

Linear Search Time complexity : $O(n)$

Binary Search Time complexity : $O(\log n)$

Binary Search is more suitable because its time complexity is $O(\log n)$, making product searches much faster than Linear Search's $O(n)$.

Exercise 7: Financial Forecasting

1.Explain the concept of recursion and how it can simplify certain problems?

Recursion is method calling itself. it simplifies problems by breaking a large problem into smaller versions of the same problem.

```
Week_1 > Design_and_principal > J FinancialForecast.java > FinancialForecast
1 public class FinancialForecast {
2
3     public static double predictFutureValue(
4         double currentValue,
5         double growthRate,
6         int years) {
7
8         // Base case
9         if (years == 0) {
10             return currentValue;
11         }
12
13         // Recursive call
14         return predictFutureValue(
15             currentValue,
16             growthRate,
17             years - 1)
18             * (1 + growthRate);
19     }
20
21     Run | Debug
22     public static void main(String[] args) {
23
24         double currentValue = 10000;
25         double growthRate = 0.10; // 10%
26         int years = 3;
27
28         double futureValue =
29             predictFutureValue(
30                 currentValue,
31                 growthRate,
32                 years);
33
34         System.out.println(
35             "Future Value: ₹" + futureValue);
36     }
}
```

```
c:\Users\Lohitha\OneDrive\Desktop\Digital_Nurture_Cts\Week_1\Design_and_principal>java FinancialForecast
Future Value: ₹13310.000000000004
```

Time Complexity: $O(n)$

Explain how to optimize the recursive solution to avoid excessive computation.

For Large input there will be more recursive call and more memory need,for optimize it can use loops.